Block Scope vs Function Scope

Where Can I Access This Variable?

Prepared by Claude for Akshay

1. The Core Idea

Scope answers: **"Where can I access this variable?"** JavaScript has two types of scope boundaries:

```
Function Scope  ->  variable visible ANYWHERE inside the function
                    Uses: var
                    Boundary: function() { }
Block Scope     ->  variable visible ONLY inside the { } block
                    Uses: let, const
                    Boundary: any { }  (if, for, while, switch, plain {})
```

> **■ The Analogy**
>
> **Function Scope (var)** = Open-plan office — you can see everyone on the floor, no cubicle walls
> **Block Scope (let/const)** = Private offices — each room has walls, you can only see people in your room

2. Function Scope — var Leaks Out of Blocks

```
function example() {
  if (true) {
    var x = 10;     // created inside if
  }
  for (var i = 0; i < 3; i++) {
    var y = 20;     // created inside for
  }
  {
    var z = 30;     // created inside plain block
  }
  // ALL leaked out!
  console.log(x);   // 10  (leaked from if)
  console.log(i);   // 3   (leaked from for)
  console.log(y);   // 20  (leaked from for)
  console.log(z);   // 30  (leaked from block)
}
example();
console.log(x);     // ReferenceError (stopped at function wall)
```

> **■ var Ignores ALL Block Boundaries**
>
> `var` treats the entire function as one big open room. `if`, `for`, `while`, `switch`, and plain `{ }` are invisible to it. The ONLY wall `var` respects is the function boundary.

3. Block Scope — let/const Stays In Its Block

```
function example() {
  if (true) {
    let x = 10;
    const y = 20;
  }
  for (let i = 0; i < 3; i++) {
    let z = 30;
  }
  console.log(x);  // ReferenceError (stayed in if)
```

```
  console.log(y);  // ReferenceError (stayed in if)
  console.log(i);  // ReferenceError (stayed in for)
  console.log(z);  // ReferenceError (stayed in for)
}
```

✔ Every { } Is a Wall

`let` and `const` treat every `{ }` as a separate room. Variables created inside a block are **destroyed** when the block exits. Inner blocks CAN see outer variables (scope chain goes up), but outer blocks CANNOT see inner block-scoped variables.

4. Side-by-Side Proof — Every Block Type

Block Type	var (Function Scope)	let/const (Block Scope)
if (true) { }	Leaks out	Stays inside
for (...) { }	Leaks out (i survives)	Stays inside
while (...) { }	Leaks out	Stays inside
switch (...) { }	Leaks out	Stays inside
Plain { }	Leaks out	Stays inside
try { } catch { }	Leaks out	Stays inside
function() { }	Stays inside (only wall!)	Stays inside

```
// Quick proof - run in console
function proof() {
  if (true) { var a = 1; let b = 2; }
  for (var c = 0; c < 1; c++) { let d = 4; }
  { var e = 5; let f = 6; }
  console.log(a);  // 1  (var leaked)
  console.log(c);  // 1  (var leaked)
  console.log(e);  // 5  (var leaked)
  // b, d, f -> all ReferenceError (let stayed in block)
}
```

5. Real Bugs Caused by Function Scope

Bug 1: Accidental Variable Overwrite

```
function processUser(user) {
  var name = user.name;
  if (user.isAdmin) {
    var name = "ADMIN: " + user.name;  // OVERWRITES outer!
    console.log(name);  // "ADMIN: Akshay"
  }
  console.log(name);  // "ADMIN: Akshay" <- BUG! Original lost
  // Both 'var name' are the SAME variable (function-scoped)
  // FIX: use let — creates separate variable in block
  // let name = "ADMIN:..." would NOT overwrite the outer one
}
```

Bug 2: The Classic Loop Closure Bug

```
// var: ONE shared variable -> all callbacks see final value
for (var i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 5, 5, 5, 5, 5  (all see i=5)
// let: NEW variable per iteration -> each callback has its own
for (let i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 100);
}
```

```
// Output: 0, 1, 2, 3, 4
```

★ Why let Fixes Loop Closures

With `var`: one `i` in the VariableEnvironment, shared by all closures. With `let`: a **new LexicalEnvironment** is created per iteration, each with its own `i`. Each closure captures a **different** environment.

Bug 3: Loop Variable Leak

```
function findItem(items, target) {
  for (var i = 0; i < items.length; i++) {
    if (items[i] === target) break;
  }
  // i leaked! Accidentally used later...
  console.log(`Found at ${i}`); // works but fragile
  var next = items[i + 1];      // could be out of bounds
  // With let: i would be ReferenceError here
  // Immediately catching the bug at development time!
}
```

6. Nested Blocks — Scope Goes UP, Not Down

```
function nested() {
  let level1 = "L1";
  {
    let level2 = "L2";
    console.log(level1);  // "L1"  (can see outer)
    {
      let level3 = "L3";
      console.log(level1);  // "L1"  (can see outer outer)
      console.log(level2);  // "L2"  (can see outer)
    }
    // console.log(level3); // ReferenceError (can't see inner)
  }
  // console.log(level2);   // ReferenceError (can't see inner)
}
```

■ Scope Chain Rule

Inner blocks CAN see outer variables — the scope chain goes UP to parent scopes.
Outer blocks CANNOT see inner variables — walls block downward access.
This applies to both function scope and block scope.

7. Special Cases

try/catch — Own Block Scope

```
try {
  let x = 10;
  throw new Error("oops");
} catch (error) {
  // error is block-scoped to catch
  console.log(error.message);  // "oops"
  // console.log(x);           // ReferenceError (try block)
}
// console.log(error);         // ReferenceError
```

Global var vs let — window Property Difference

```
var a = 1;    // window.a = 1      (becomes global property)
let b = 2;    // window.b = undefined! (NOT on window)
```

```
const c = 3;  // window.c = undefined! (NOT on window)
console.log(window.a);  // 1
console.log(window.b);  // undefined!
```

Function Declarations in Blocks — Avoid!

```
// INCONSISTENT behavior across browsers!
if (true) {
  function greet() { return "Hello"; }
}
greet(); // works in some engines, breaks in others
// FIX: use function expression with const
if (true) {
  const greet = function() { return "Hello"; };
}
// greet(); // ReferenceError (predictable!)
```

8. React Practical Examples

```
// Block scoping makes conditional rendering safe
function Dashboard({ user }) {
  if (user.role === "admin") {
    const tools = loadAdminTools();  // block-scoped
    return <AdminPanel tools={tools} />;
  }
  if (user.role === "editor") {
    const tools = loadEditorTools();  // different block, no clash!
    return <EditorPanel tools={tools} />;
  }
  return <ViewerPanel />;
  // No variable name collisions between blocks!
}
```

```
// .map() works perfectly because arrow creates block scope
function TodoList({ todos }) {
  return todos.map((todo, index) => (
    // Each iteration: new block scope for 'todo' and 'index'
    <div key={index}>
      {todo.text}
      <button onClick={() => deleteTodo(index)}>X</button>
    </div>
  ));
}
```

9. Complete Comparison Table

Feature	Function Scope (var)	Block Scope (let/const)
Boundary	function() { } only	Any { }
Leaks from if/for?	YES	NO
Hoisting	Hoisted as undefined	Hoisted but TDZ
Re-declaration	Allowed	SyntaxError
Loop closures	One shared variable	New copy per iteration
Global behavior	Becomes window.x	NOT on window
Internals	VariableEnvironment	LexicalEnvironment
Can overwrite outer?	Yes (same scope!)	No (new variable)
ES Module behavior	Module-scoped	Block-scoped
Use in 2026?	Never (legacy only)	Always

10. Cheat Sheet

✔ 7 Quick Rules

1. `var` = function is the wall, blocks are invisible
2. `let/const` = every `{ }` is a wall
3. Inner can see outer (scope chain goes UP)
4. Outer cannot see inner (walls block downward)
5. Loop + `var` = one variable shared by all iterations
6. Loop + `let` = new variable per iteration
7. Default to `const`, use `let` when needed, **never use var**

★ Interview One-Liner

"Function scope means `var` is visible throughout the entire function regardless of blocks. Block scope means `let` and `const` are confined to the nearest `{ }` block. This is why `let` fixes the classic loop closure bug — each iteration creates a new block scope with its own copy of the variable, while `var` shares one variable across all iterations."